

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA Nº 08

NOMBRE COMPLETO: Luis Daniel Salazar Islas

Nº de Cuenta: 320335163

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 06

SEMESTRE 2026-1

FECHA DE ENTREGA LÍMITE: 25/10/2025

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Como todos los ejercicios involucraban prender y apagar luces decidí implementar una serie de funciones que automatizaran el proceso.

Vectores que contendrán los nombres de cada SpotLight y PointLight creado, de esta manera realizamos identificación por nombre en las búsquedas.

```
std::vector<std::string> spots(MAX_SPOT_LIGHTS),points(MAX_POINT_LIGHTS);
```

Funciones de búsqueda de la luz por nombre, regresa el índice del arreglo que lo contiene

```
int searchPoint(std::string id) {
    for (int i = 0;i < MAX_POINT_LIGHTS;i++) {
        if (points[i] == id) return i;
    }
    printf("NO SE ENCONTRO LA LUZ PointLight\n");
    return -1;
}
```

```
int searchSpot(std::string id) {
    for (int i = 0;i < MAX_SPOT_LIGHTS;i++) {
        if (spots[i] == id) return i;
    }
    printf("NO SE ENCONTRO LA LUZ SpotLight\n");
    return -1;
}
```

Funciones para apagar las luces. La idea es que intercambiemos la luz que nosotros queramos eliminar con la posición de la última luz activa, de tal forma que al restar el contador esta luz se elimine. Primero se busca la luz a eliminar entre las luces activas, si no está, termina la función.

```cpp
void turnOffSpot(std::string id, unsigned int& spotLightCount) {
    if (spotLightCount <= 0) {
        return;
    }
    int idx=-1;
    for(int i=0;i<spotLightCount;i++){
        if(spots[i]==id){
            idx=i;
            break;
        }
    }
    if (idx == -1) return;
    std::swap(spotLights[idx], spotLights[--spotLightCount]);
    std::swap(spots[idx], spots[spotLightCount]);
}
```

```cpp
void turnOffPoint(std::string id, unsigned int& pointLightCount)
{
    if (pointLightCount <= 0) {
        return;
    }
    int idx=-1;
    for(int i=0;i<pointLightCount;i++){
        if(points[i]==id){
            idx=i;
            break;
        }
    }
    if (idx == -1) return;
    std::swap(pointLights[idx], pointLights[--pointLightCount]);
    std::swap(points[idx], points[pointLightCount]);

}
```

Funciones para activar luces apagadas. En este caso, buscamos entre las luces inactivas la luz que deseamos activar y la intercambiamos con la primera luz inactiva, de tal forma que al intercambiar y sumar el contador se añadiera al shader.

```
void turnOnPoint(std::string id, unsigned int& pointLightCount) {
    if (pointLightCount >= MAX_POINT_LIGHTS) {
        return;
    }
    int idx =-1;
    for(int i=pointLightCount;i<MAX_POINT_LIGHTS;i++){
        if(points[i]==id){
            idx=i;
            break;
        }
    }
    if (idx == -1) return;
    std::swap(pointLights[idx], pointLights[pointLightCount]);
    std::swap(points[idx], points[pointLightCount]);
    pointLightCount++;
}
```

```
void turnOnSpot(std::string id, unsigned int& spotLightCount) {
    if (spotLightCount >= MAX_SPOT_LIGHTS) {
        return;
    }
    int idx =-1;
    for(int i=spotLightCount;i<MAX_SPOT_LIGHTS;i++){
        if(spots[i]==id){
            idx=i;
            break;
        }
    }
    if (idx == -1) return;
    std::swap(spotLights[idx], spotLights[spotLightCount]);
    std::swap(spots[idx], spots[spotLightCount]);
    spotLightCount++;
}
```

Para la cantidad de luces spot y point requeridas se hizo un ajuste en las cantidades máximas

```
#ifndef COMMONVALS
#define COMMONVALS
#include "stb_image.h"

const int MAX_POINT_LIGHTS = 2;
const int MAX_SPOT_LIGHTS = 5;
#endif
```

```
const int MAX_POINT_LIGHTS = 2;
const int MAX_SPOT_LIGHTS = 5;
```

Ejercicio 1

Se nos pide agregar una luz spotLight al cofre, la cual debe ajustar su dirección según la rotación del cofre.

Se crea una luz spotlight

```
//luz cofre
spotLights[1] = SpotLight(0.0f, 1.0f, 0.0f,
    1.0f, 2.0f,
    5.0f, 10.0f, 0.0f,
    1.0f, 0.0f, 0.0f,
    1.0f, 0.01f, 0.01f,
    15.0f);
spots[spotLightCount] = "linternaCofre";
spotLightCount++;
```

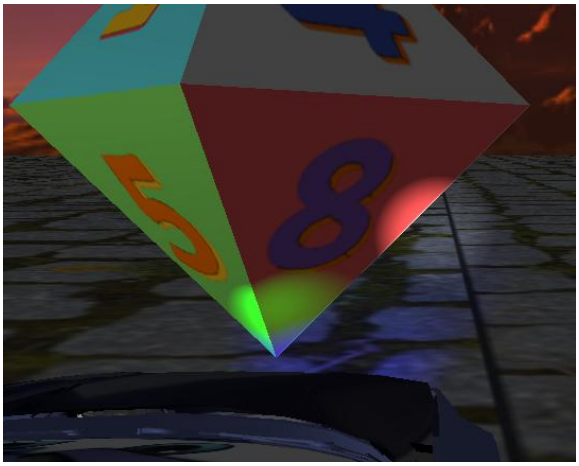
Para darle la dirección, lo que hago es crear la matriz de rotación por medio de rotate y después la multiplico con el vector de dirección de la luz que en este caso es de -x. La posición se calcula fácilmente con jerarquía y mandos los valores calculados a SetFlash. Dado la lógica de prendido y apagado de luces ya no se puede asegurar el orden de las luces y por ello siempre se realiza una búsqueda de la luz en este caso, linternaCofre.

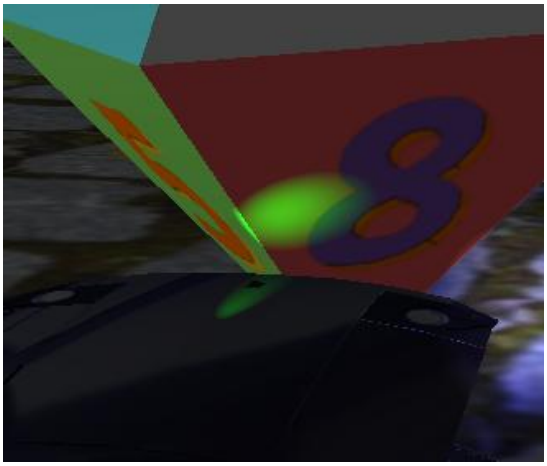
```
//luz cofre
model = modelaux;
model = glm::translate(model,glm::vec3(0.0f,0.0f,-1.0f));
idx=search("linternaCofre", spotLightCount);
rot = glm::rotate(glm::mat4(1.0), glm::radians(-mainWindow.getarticulacion5()), glm::vec3(0.0f, 0.0f, 1.0f))*glm::vec4(-1.0f,0.0f,0.0f,0.0f);

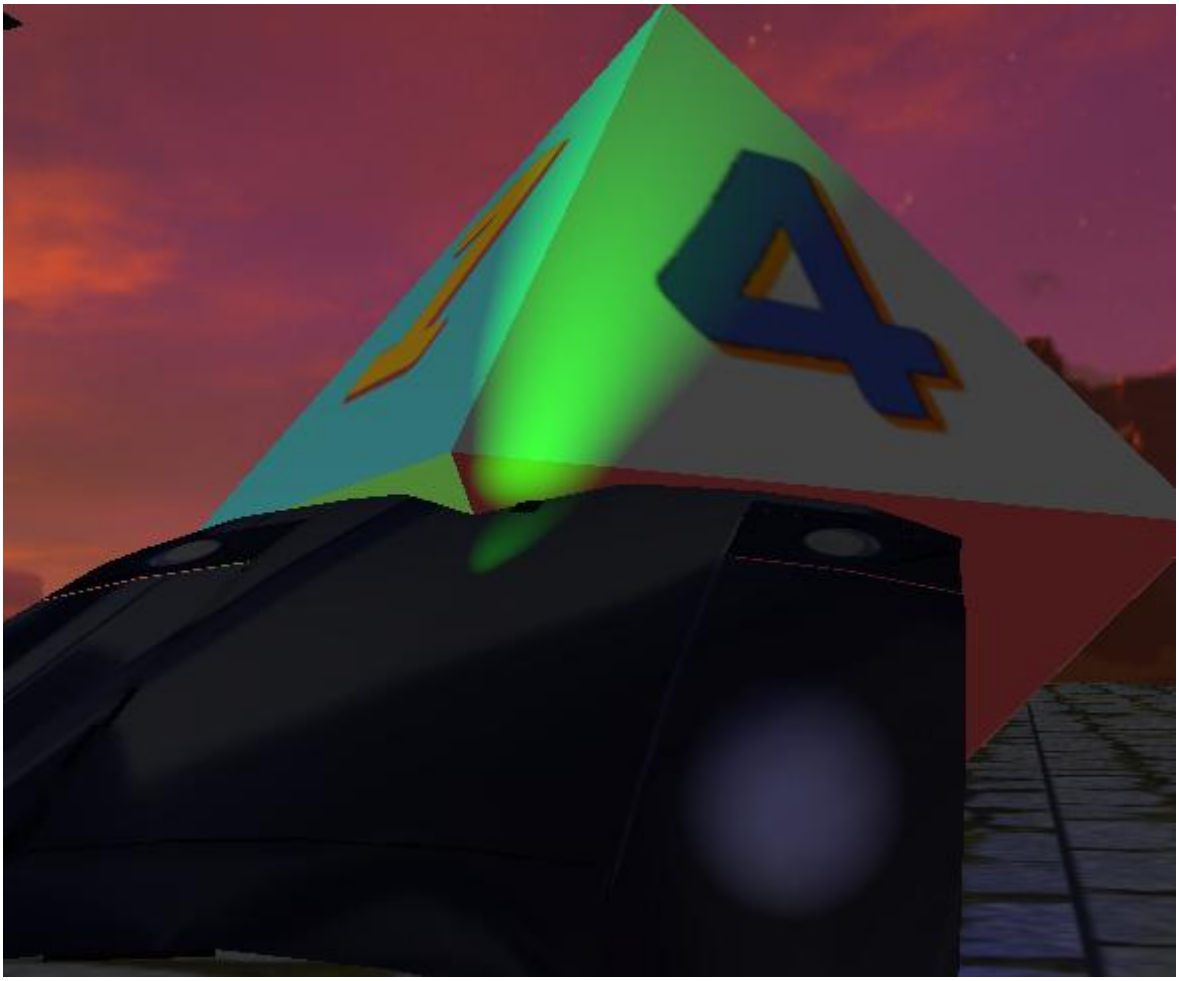
spotLights[idx].SetFlash(model[3],glm::vec3(rot[0],rot[1],rot[2]));
```

Para probar el funcionamiento utilice el dado como muro frontal.

Ejecución







Ejercicio 2

Para el ejercicio se nos pide colocar dos luces spotlights al frente y atrás del coche y que se alternen dependiendo del movimiento del coche. Si el coche se mueve hacia adelante se prende la luz frontal y en reversa la luz trasera.

Primero definí en window.h una bandera para indicar que luz estará prendida.

```
    GLfloat forward() { return front; }
    ~Window();
private:
    GLFWwindow *mainWindow;
    GLfloat rotax, rotay, rotaz, articulacion
    GLint width, height;
    bool isOn,antorch,raygun;

    bool front;
```

```
front = true;
```

Dependiendo de la tecla presionada (avanzar o retroceder) se activa la bandera para indicar el movimiento del coche.

```
if (key == GLFW_KEY_Y)
{
    theWindow-> muevex -= 1.0;
    theWindow->front = true;
}
if (key == GLFW_KEY_U)
{
    theWindow-> muevex += 1.0;
    theWindow->front = false;
}
```

En el main se enciende y apaga la luz de acuerdo al valor de la bandera.

```
if (mainWindow.forward()) {
    turnOnSpot("faroDelantero",spotLightCount);
    turnOffSpot("faroTrasero", spotLightCount);
}
else {
    turnOnSpot("faroTrasero", spotLightCount);
    turnOffSpot("faroDelantero", spotLightCount);
}
```

La posición de los faros se hace con jerarquía

```
//Faro

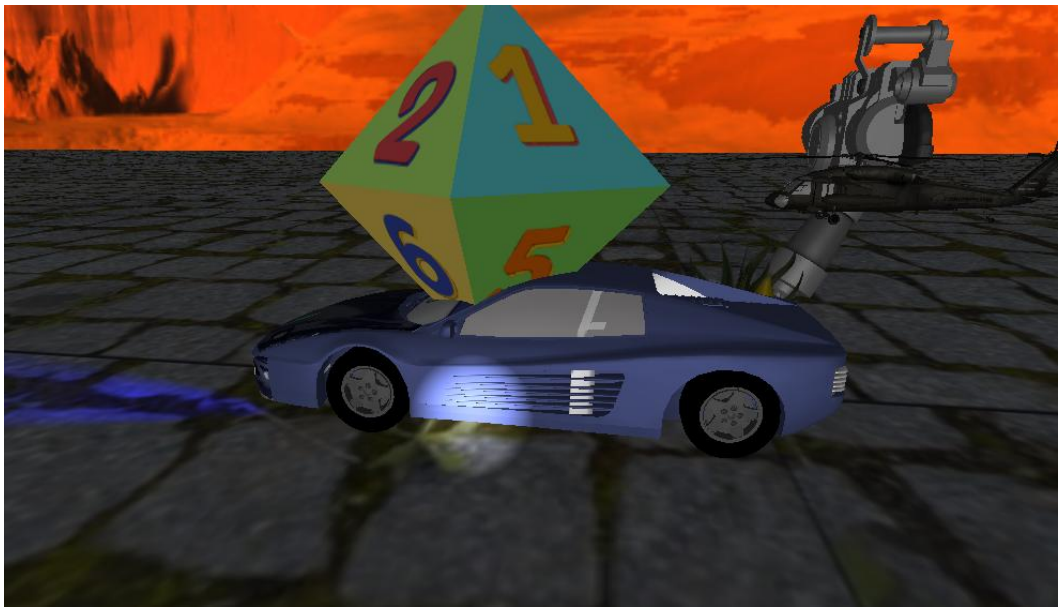
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -1.0f));
p = glm::vec3(model[3]);
idx = searchSpot("faroDelantero");
spotLights[idx].SetPos(p);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 1.0f));
p = glm::vec3(model[3]);
idx = searchSpot("faroTrasero");
spotLights[idx].SetPos(p);
```

Ejecución

En reversa



Avanzando



Ejercicio 3

Para el tercer ejercicio se pide importar otro modelo y ligarle una luz puntual. Tanto para el nuevo modelo como la lámpara del ejercicio de clase se debe establecer una tecla que prenda y apague cada luz independientemente.

Escogí como modelo una pistola de rayos de ciencia ficción. La luz representa el haz de energía que se acumulan en estas armas al disparar.



Se crean las banderas de cada luz

```
    bool AntorchOn() { return antorch; }
    bool RayGunOn() { return raygun; }

    GLfloat forward() { return front; }
    ~Window();
private:
    GLFWwindow *mainWindow;
    GLfloat rotax, rotay, rotaz, articulacion1, articul
    GLint width, height;
    bool isOn,antorch,raygun;
```

Se le asignan las teclas

```
if (key == GLFW_KEY_B && action == GLFW_RELEASE) {
    theWindow->antorch = !theWindow->antorch;
}

if (key == GLFW_KEY_N && action == GLFW_RELEASE) {
    theWindow->raygun = !theWindow->raygun;
}
```

En el main prendo y apago las luces dependiendo del valor de la bandera.

```
shaderList[0].SetDirectionalLight(&mainLight);

if (mainWindow.AntorchOn()) {
    turnOnPoint("antorcha", pointLightCount);
}
else {
    turnOffPoint("antorcha", pointLightCount);
}

if (mainWindow.RayGunOn()) {
    turnOnPoint("laser", pointLightCount);

}
else {
    turnOffPoint("laser", pointLightCount);
}
```

Código para renderizar la pistola y su luz mediante jerarquía

```
//Raygun

model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(6.0f, 5.0f, -6.0));
modelaux = model;
model = glm::scale(model, glm::vec3(0.1f, 0.1f, 0.1f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
raygun.RenderModel();

//Rayo raygun
model = modelaux;
model = glm::translate(model, glm::vec3(-2.6, 1.4f, -5.7));

p = glm::vec3(model[3]);
idx = searchPoint("laser");
pointLights[idx].SetPos(p);
```

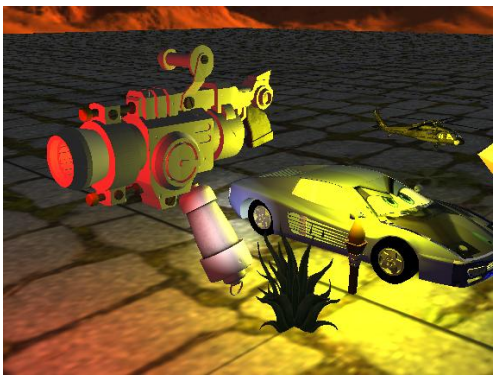
Código para renderizar la antorcha y su luz ligada por jerarquía

```
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, -2.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(40.0f, 40.0f, 40.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
lampara.RenderModel();

model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 6.9f, 0.0f));
p = glm::vec3(model[3]);
idx = searchPoint("antorcha");
```

<u>Ejecución (Todas las combinaciones)</u>



2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

La mayoría de los problemas surgieron al diseñar las funciones de prendido y apagado. Sobre todo, abarcar casos especiales como intentar prender una luz ya prendida o intentar apagar una ya apagada. Al final, la lógica se simplificó a lo mostrada, pero llevo un tiempo.

3.- Conclusión:

a. Los ejercicios del reporte: Complejidad, Explicación.

Los ejercicios me parecieron interesantes porque en este caso hice mayor use de lógica para completar los ejercicios, especialmente al calcular la nueva dirección de la luz en el cofre y al diseñar las funciones para prender y apagar luces en el orden que quiera. Si bien requerían de lógica, no siento que fuera una lógica muy compleja.

b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias para mejorar desarrollo de la práctica

En general, toda la explicación me pareció correcta, la parte principal de agregar y borrar luces es jugar con las últimas posiciones por lo que esa idea fue clave para el diseño de las funciones. Para la realización de la práctica no requerí explicación más allá de lo que se dio en clase.

c. Conclusión

1. Bibliografía en formato APA

- Sketchfab. (n.d.). *Fountain - Download Free 3D model by AnimaZeal (@Animazeal_13)*. https://sketchfab.com/3d-models/ray-gun-cartoon-aebdb6a57bcd4b58ae95f6f0944d922e